

FAQ

Setup Wizard FAQ

Answers to the questions that come up after running `scripts/setup-agents-wizard.sh`.

- What the wizard installs: [README.md](#)
- Symptom -> diagnosis -> fix: [TROUBLESHOOTING.md](#)
- Undoing a run: [SAFETY-AND-ROLLBACK.md](#)
- The three post-install scripts: [OPERATIONAL-SCRIPTS.md](#)
- The steps the wizard cannot perform: [ACTION-REQUIRED.md](#)
- Whether an index can be trusted: [INDEX-CORRUPTION-RECONCILIATION.md](#)

Everything below describes the script as it currently exists in this repository.

Why isn't lumen found after running the wizard?

Because the shell you ran the wizard *from* never re-read its startup files.

The wizard writes the launcher to `~/.local/bin/lumen` and appends a managed block to `~/.bashrc` and `~/.bash_profile` that puts `~/.local/bin` on `PATH`. Those blocks only take effect in shells started *after* the wizard ran. The `export PATH=...` the wizard does at the end of `configure_lumen_shell` applies to the wizard's own process and dies with it.

Fix — pick one:

```
exec bash -l          # replace the current shell with a fresh
                      login shell
. ~/.bashrc           # or just re-source it
```

Then verify:

```
command -v lumen      # -> /home/<you>/.local/bin/
                      lumen
lumen version         # -> e.g. 0.0.41
grep -c '>>> lumen semantic search' ~/.bashrc    # -> 1
grep -c '>>> user bin on PATH'      ~/.bash_profile # -> 1
```

The wizard performs this same check itself (`verify_lumen`) using `bash -lc 'command -v lumen'` and prints `lumen not on PATH` in a login shell - open a new terminal to pick it up. when it fails.

Two things that will still leave `lumen` missing:

Situation	Why	What to do
You use <code>zsh</code> / <code>fish</code>	The wizard only writes <code>~/.bashrc</code> and <code>~/.bash_profile</code>	Add <code>~/.local/bin</code> to <code>PATH</code> in your own rc file
Wrapper exists but nothing resolves it	The Lumen <i>plugin</i> is not installed	Run <code>~/.local/bin/lumen version</code> directly — see TROUBLESHOOTING #1

Why does `lumen` work in my terminal but not in `cron` / `ssh host 'cmd'` / a `systemd` user unit?

Because those contexts do not run an interactive shell, and the usual place `~/.local/bin` gets added to `PATH` is guarded against exactly that.

A stock `~/.bashrc` starts with:

```
case $- in
  *i*) ;;
  *) return;;
esac
```

Anything appended to `~/.bashrc` — including the wizard’s Lumen block — is *below* that guard, so a non-interactive shell returns before reaching it. Meanwhile a login shell starts from the PATH that `/etc/profile` builds, which does not contain `~/.local/bin`.

That is why the wizard writes a **second**, smaller block into `~/.bash_profile`:

```
# >>> user bin on PATH (managed by Claude Code) >>>
case ":$PATH:" in *"$HOME/.local/bin:*") ;; *) export
PATH="$HOME/.local/bin:$PATH";; esac
# <<< user bin on PATH <<<
```

`~/.bash_profile` is read by **login** shells regardless of interactivity, so this covers `bash -lc '<cmd>'`, console/SSH logins, and any unit or job that starts a login shell.

It does **not** cover shells that are neither login nor interactive. For those, do not rely on PATH at all:

Context	Reliable fix
crontab	Put <code>PATH=/home/<you>/.local/bin:/usr/bin:/bin</code> at the top of the crontab, or call <code>\$HOME/.local/bin/lumen</code>
systemd user unit	<code>Environment=PATH=%h/.local/bin:/usr/bin:/bin</code> , or use the absolute path in <code>ExecStart=</code>
ssh host 'cmd'	<code>ssh host 'bash -lc "lumen ..."'</code> or <code>ssh host '~/.local/bin/lumen ...'</code>
An agent spawning an MCP server	Already handled: the wizard writes the absolute wrapper path into every MCP config

That last row is the same problem: agents spawn MCP servers from non-interactive shells, so `configure_mcp_for_agent` writes `"command": "/home/<you>/.local/bin/lumen"` instead of a bare lumen.

Why is Lumen not installed from npm?

Because there is no such npm package. `@qoomon/lumen` does not exist — the registry returns 404.

An earlier version of the wizard had it in the global `npm install -g ...` line with a trailing `|| true`, which swallowed the 404 silently: `lumen` was never installed, but the final summary still looked healthy.

Lumen ships as a **Claude Code plugin binary**, at a path like:

```
~/.claude-shared/plugins/cache/<marketplace>/lumen/<version>/bin/  
lumen-<os>-<arch>
```

So the wizard installs nothing from npm for Lumen. It installs a wrapper on `PATH` (ensure `_lumen`, Step 3) that locates that plugin binary at call time. If the plugin itself is missing, install it from inside Claude Code with `/plugin`.

The test suite guards against a regression here — assertion **A2** fails the build if the string `@qoomon/lumen` reappears anywhere in the wizard.

Why is there a wrapper script instead of a symlink?

Because the plugin path contains a **version** component that changes on every update:

```
~/.claude-shared/plugins/cache/<marketplace>/lumen/0.0.41/bin/  
lumen-linux-amd64
```

^^^^^

A symlink (or a hardcoded `PATH` entry) into that directory breaks the moment the plugin updates and the old version directory is removed — and it breaks *silently*, because the dangling link still looks like an installed `lumen` until you run it.

`~/.local/bin/lumen` is a small bash script that resolves the binary on every invocation:

1. If `LUMEN_BIN` is set, use it (and exit 127 if it is not executable).
2. Map the host to the plugin naming convention: `linux/darwin x amd64/arm64`.

3. Glob `~/.claude-shared/plugins/cache/*/lumen/*/bin/lumen-<os>-<arch>`, falling back to `~/.claude*/plugins/cache/*/lumen/*/bin/lumen-<os>-<arch>`.
4. Pick the **highest** version with `sort -V`, then exec it.

`sort -V` (version sort) rather than a plain lexical sort is load-bearing:

Sort	Order	Winner
lexical (sort)	0.0.100, 0.0.41, 0.0.9	0.0.9 — wrong
version (sort -V)	0.0.9, 0.0.41, 0.0.100	0.0.100 — correct

Once the plugin passes 0.0.99, a lexical sort would pin you to an old binary forever. Test **B3** installs fake 0.0.9, 0.0.41 and 0.0.100 binaries and asserts the wrapper runs 0.0.100.

To bypass resolution entirely (custom build, a binary outside the plugin cache):

```
export LUMEN_BIN=/path/to/lumen-linux-amd64
```

The wrapper is safe to delete; re-run the wizard (or just `install_lumen_wrapper`) to restore it.

Why does the wizard not set LUMEN_EMBED_MODEL?

Because pinning it desynchronizes two consumers that must agree.

The `lumen CLI` and the `lumen stdio MCP servers` running inside each agent share the same **built-in defaults**. If you export `LUMEN_EMBED_MODEL` in `~/.bashrc`, only the processes that inherit that environment get your value — and after a plugin update whose default model changed, the CLI and the agent-spawned server can disagree.

A model mismatch is not a hard error. Lumen keys its index by the embedding model, so a second model means Lumen builds a **second complete index for every project**: double the embedding time, double the disk usage, and searches that hit a half-built index.

For this reason the wizard writes the tuning knobs into `~/.bashrc` **commented out**:

```
#export LUMEN_BACKEND=ollama #
ollama | lmstudio
#export LUMEN_EMBED_MODEL=ordis/jina-embeddings-v2-base-code
#export OLLAMA_HOST=http://localhost:11434
#export LUMEN_REINDEX_TIMEOUT=600 # seconds; raise for very
large repositories
#export LUMEN_FRESHNESS_TTL=300 # seconds before an index is
treated as stale
#export LUMEN_LOG_LEVEL=info # debug | info | warn | error
```

The wizard still *uses* `ordis/jina-embeddings-v2-base-code` internally — that is the model it pulls into `ollama` (`ensure_ollama`) and the one it reports on in the summary. It just does not force it on your shell.

If you genuinely need a different model, prefer the per-command flag so the change is scoped and visible:

```
lumen index /path/to/repo -m <model>
lumen search "... " -p /path/to/repo -m <model>
```

and expect a full extra index to be built for that model.

Is it safe to re-run the wizard?

Yes. Re-running is the supported way to repair a broken setup.

Mutation	Why re-running is safe
~/.bashrc, ~/.bash_profile	Blocks are marker-delimited. <code>strip_managed_block</code> removes the old block with <code>awk</code> before the new one is appended — content outside the markers is untouched, and the file's mode is preserved
PATH line inside those blocks	Guarded by case <code>":\$PATH:" in *":\$HOME/.local/bin:">*</code> ; ; ... — sourcing twice cannot add a second entry
Agent MCP configs	<code>jq</code> merges by key: existing keys are replaced, unrelated keys are preserved, nothing is duplicated. Claude Code is skipped outright when <code>claude mcp get lumen</code> already succeeds

Mutation	Why re-running is safe
~/.claude/settings.json	The Glyphdown entry is appended to <code>.hooks.PreToolUse / .hooks.PostToolUse</code> only when <code>[.hooks[\$ev]</code> <code>[]?.hooks[]?.command] \ </code> <code>index("glyphdown")</code> finds nothing — so re-runs neither duplicate it nor disturb your own hooks. The file is not touched at all when <code>WIZARD_SKIP_GLYPHDOWN_HOOK</code> is set, or when <code>glyphdown</code> is not installed
Telemetry opt-out	Same marker-delimited strip- then-append as the other ~/.bashrc block (test H2), and ~/.qwen/settings.json is rewritten only when <code>.usageStatisticsEnabled</code> is not already false
Global CLI installs	Guarded by <code>check_command</code>: a tool already on <code>PATH</code> is left alone, never reinstalled or upgraded
Every file it edits	Backed up first as <code><file>.bak.YYYYYMMDDHHMMSS</code>, and recorded in a rollback manifest — see How do I completely uninstall
~/.local/bin/lumen	Backed up, then rewritten from the embedded template
Cloned repos (SuperSpec, marketplaces)	Skipped when already present
A registered SuperSpec submodule	Detected via <code>[[-e</code> <code>"\$sub_path/.git"]]</code> — a gitlink is a <i>file</i>, not a directory — and never deleted

That last row was a real data-loss bug: the old check `[[ -d ../.git ]]` is always false for a submodule, so the `rm -rf` beneath it deleted the checkout on every run. Test **E1** is the regression guard and asserts a canary file survives.

The idempotency claims are enforced by tests **C3**, **C4**, **C8**, **C10** and **D5** (three consecutive runs, exactly one block; two runs, exactly two MCP servers).

What re-running does **not** do: it never re-indexes anything **unless you set `WIZARD_INDEX_PROJECT`** (see [Does the wizard index my project?](#)), and it never deletes an index.

Where does Lumen store indexes and how much space do they use?

In `~/.local/share/lumen/`, one directory per project:

```
~/.local/share/lumen/
├── 74e360aac96f417e/
│   ├── index.db          <- SQLite database with the embeddings
│   ├── index.db-wal
│   ├── index.db-shm
│   └── index.db.lock
└── ...
```

Directory names are opaque hashes; the project path is recorded *inside* the database, which is what lets `lumen purge <path>` match the right one.

They get big. Size scales with the amount of indexed code, and every project you have ever searched from an agent leaves one behind. A machine with a few hundred indexed projects can easily hold gigabytes — an observed example: 2.3 GB across 764 project directories, with the largest single project at 666 MB.

Inspect it:

```
du -sh ~/.local/share/lumen          # total
du -sh ~/.local/share/lumen/* | sort -h | tail -10 # biggest offenders
ls -l ~/.local/share/lumen | wc -l    # how many projects
```

Reclaim it:

```
lumen purge /path/to/project # remove ONE project's index
                             (path is normalized to its git root)
lumen purge                  # remove EVERY index – irreversible
```


Bare `lumen purge` takes no confirmation prompt. Nothing is lost permanently (indexes rebuild on the next `lumen index` or search), but rebuilding everything costs the full embedding time again, which for hundreds of projects is hours.

Two details worth knowing:

- Indexes created by older binaries did not record `project_path` and cannot be matched by path; only a bare `lumen purge` clears those.
 - Purging a project while an indexer is running for it makes that indexer log a write error and exit — re-run `lumen index <path>` afterwards.
-

How do I completely uninstall what the wizard added?

Use the rollback tool. There is no `--uninstall` flag on the wizard, but every mutation it makes is recorded in a manifest and `scripts/rollback-agents-wizard.sh` replays it in reverse:

```
./scripts/rollback-agents-wizard.sh --list
# which runs can be undone
./scripts/rollback-agents-wizard.sh --dry-run
# print the plan, change nothing
./scripts/rollback-agents-wizard.sh --run-actions --yes #
# apply, including the npm, Speckit and claude mcp undo
# commands
exec bash -l # pick
# up the restored PATH
```

That restores every modified file **byte-exactly**, with its original permissions; deletes every file the wizard created; and runs the recorded undo commands (`npm uninstall -g ...`, `uv tool uninstall specify-cli`, `claude mcp remove lumen -s user`). Without `--run-actions` those commands are printed for you to run yourself. The rollback is itself reversible — current state is copied into `<session>/pre-rollback-<UTC>/` before anything is touched.

Three limits worth knowing before you rely on it:

- A session is **one wizard run**. Roll back several runs newest-first with `--session <id>`.
- Work delegated to third parties is not in the manifest: Bun, ashlr, WOZCODE, Ollama and its model, the two `git cloned` marketplaces,

and the project-level `.specify/ / submodules/`
`superspec / .codegraph/`.

- `--run-actions` with `no --component` **will** uninstall SpecKit, because the `uv` install now records `uv tool uninstall specify-cli` under component `speckit`. Scope the rollback (`-c npm`, `-c claude`, ...) if you want to keep it.

For those, and for the case where the manifest is gone, remove things in this order, skipping anything you still want.

1. Lumen launcher, completions and backups

```
rm -f ~/.local/bin/lumen ~/.local/bin/lumen.bak.*
rm -f ~/.local/share/bash-completion/completions/lumen
```

2. The managed shell blocks — reuse the wizard's own remover in library mode, so the markers and file permissions are handled correctly:

```
cd /path/to/this/repo
SETUP_WIZARD_LIB_ONLY=1 . scripts/setup-agents-wizard.sh
strip_managed_block "$HOME/.bashrc" "$LUMEN_BLOCK_START"
"$LUMEN_BLOCK_END"
strip_managed_block "$HOME/.bashrc" "$PRIVACY_BLOCK_START"
"$PRIVACY_BLOCK_END"
strip_managed_block "$HOME/.bash_profile" "$USERBIN_BLOCK_START"
"$USERBIN_BLOCK_END"
```

There are three blocks: `~/.bashrc` holds both the Lumen block and the telemetry opt-out block, `~/.bash_profile` the user-bin block.

3. MCP server entries, per agent — each at its real location, and note that Opencode uses `.mcp`, not `.mcpServers`:

```
claude mcp remove lumen -s user                                # Claude Code
    - through its own CLI

    # (this writes to the ACTIVE CLAUDE_CONFIG_DIR)
tmp=$(mktemp) && jq 'del(.mcpServers.lumen)' ~/.claude.json >
"$tmp" && mv "$tmp" ~/.claude.json

    # ^ the mirror MiMo and other inheritors read
tmp=$(mktemp) && jq
'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.kimi-
code/mcp.json > "$tmp" && mv "$tmp" ~/.kimi-code/mcp.json
tmp=$(mktemp) && jq
'del(.mcpServers.lumen, .mcpServers.codegraph)' ~/.qwen/
settings.json > "$tmp" && mv "$tmp" ~/.qwen/settings.json
tmp=$(mktemp) && jq 'del(.mcp.lumen)' ~/.config/opencode/
opencode.json > "$tmp" && mv "$tmp" ~/.config/opencode/
opencode.json
```

MiMo Code needs nothing — the wizard never wrote a config for it.

4. The Glyphdown hooks in Claude settings — they are entries inside the PreToolUse and PostToolUse arrays, so remove just those and keep your own:

```
tmp=$(mktemp) && jq '
  def strip: (. // []) | map(select((.[.hooks[]?.command] |
    index("glyphdown")) | not));
  .hooks.PreToolUse |= strip | .hooks.PostToolUse |= strip
' ~/.claude/settings.json > "$tmp" && mv "$tmp" ~/.claude/
settings.json
```

5. Cloned marketplace repositories

```
rm -rf ~/.claude/plugins/marketplaces/Sagargupta16/claude-cost-
optimizer
rm -rf ~/.claude/plugins/marketplaces/JCodesMore/jcodesmore-
plugins
```

6. Global CLI tools (note: no Lumen package — it was never installed from npm, and Kimi / Opencode / MiMo Code are never npm-installed either)

```
npm uninstall -g @colbymchenry/codegraph glyphdown @qwen-code/
qwen-code
uv tool uninstall specify-cli
```

6b. Telemetry settings — the ~/.bashrc block goes with step 2 above; the rest:

```
tmp=$(mktemp) && jq 'del(.usageStatisticsEnabled)' ~/.qwen/
settings.json > "$tmp" && mv "$tmp" ~/.qwen/settings.json
codegraph telemetry --help # CodeGraph's own switch; state
lives in ~/.codegraph/telemetry.json
```

7. Lumen indexes — see [Where does Lumen store indexes](#). lumen purge before you delete the wrapper, or just `rm -rf ~/.local/share/lumen`. If you ran Step 8, the CodeGraph database is separate and lives in the project: `rm -rf .codegraph`.

8. Embedding backend (only if nothing else uses ollama)

```
ollama rm ordis/jina-embeddings-v2-base-code
sudo systemctl disable --now ollama
```

9. Third-party installers the wizard invoked — Bun (~/.bun), which installs itself and may have added its own lines to your rc files, and the **ashlr plugin**, which is a clone rather than a binary:

```
rm -rf ~/.claude/plugins/cache/ashlr-marketplace/ashlr
```

Whatever `/plugin install` then registered inside Claude Code is Claude Code's own state — remove it there, not on disk.

10. Project-level artifacts — `submodules/superspec`, `.specify/`, `specs/`, plus the regenerated files that are deliberately *not* in the rollback manifest:

```
rm -f MANUAL-STEPS.md .lumen-reindex.log
rm -rf .ashlrcode           # the ashlr genome, if you ran /
                           ashlr:ashlr-genome-init
rm -rf .test-evidence      # test-suite output
```

Treat the rest as project content, not wizard leftovers: if the repo tracks `submodules/superspec` as a git submodule, remove it with `git submodule deinit / git rm`, not `rm -rf`.

11. The ollama Vulkan remediation, if you applied it. It is outside the manifest and has its own undo — note that undoing it **restores the defect**:

```
./scripts/ollama-vulkan-remediation.sh --rollback # needs
sudo; restarts ollama
```

11. Orphan files from older revisions — an earlier wizard wrote configs no agent reads. Delete them if they exist:

```
rm -f ~/.claude/mcp.json ~/.kimi/config.json ~/.opencode/
      config.json \
      ~/.mimo/config.json ~/.qwen-code/config.json
```

12. Timestamped backups and rollback sessions the wizard left behind

```
ls -l ~/.bashrc.bak.* ~/.bash_profile.bak.* ~/.claude/*.bak.* 2>/
dev/null
rm -rf ~/.local/share/setup-agents-wizard # the rollback
sessions – only once you are sure
```

Full detail: [SAFETY-AND-ROLLBACK.md](#).

What if I already had MCP servers configured?

They are preserved. Every edit the wizard makes to a config file is a **jq merge**, never a replacement:

Agent	What the wizard does	What survives
Kimi, Qwen Code	<code>jq '.mcpServers += {lumen, codegraph}'</code>	Every other MCP server, and every key outside <code>.mcpServers</code>
Opencode	<code>jq '.mcp //= {} \\ .mcp.lumen = {...}'</code>	Every other entry under <code>.mcp</code> , and the rest of the document
Claude Code	<code>claude mcp add-json lumen ... -s user</code> , and only when <code>claude mcp get lumen</code> fails	Everything — the CLI owns the file, and the wizard never hand-edits <code>~/.claude.json</code>
Claude hooks	<code>.hooks.PreToolUse</code> / <code>.hooks.PostToolUse</code> gain one entry, guarded by an <code>index("glyphdown")</code> check	Every hook you already had, on every event

The `+=` object merge replaces only the keys named on the right-hand side. A server you configured yourself under a different name is untouched; a server you configured *as* `lumen` or `codegraph` is overwritten — and its previous content is in the rollback manifest, so `./scripts/rollback-agents-wizard.sh -c kimi --yes` brings it back byte-exactly.

Re-running does not duplicate anything: tests **D4** and **D5** assert that foreign keys survive and that two runs still leave exactly two MCP servers.

Two related guarantees:

- **Claude Code's `~/.claude.json` is never hand-edited.** It also holds session and project state, so the wizard goes through the `claude mcp` CLI instead.
- **Nothing is written for an agent that is not installed.** Kimi is skipped unless `~/.kimi-code` exists, Qwen unless `~/.qwen` exists, and Opencode unless its config file already exists — the wizard does not create one.

Can I undo just one agent?

Yes. `--component` (short `-c`) scopes the rollback, and it is repeatable:

```
./scripts/rollback-agents-wizard.sh -c opencode --dry-run
# look first
./scripts/rollback-agents-wizard.sh -c opencode --yes      #
# remove Lumen from Opencode only
./scripts/rollback-agents-wizard.sh -c kimi -c qwen --yes  # two
# agents, nothing else
```

The components are lumen, shell, claude, kimi, opencode, qwen, npm and speckit; all is accepted and means “no filter”, which is also the default. Test **G11** asserts that a component filter leaves the other components untouched.

Claude Code is a special case: its MCP registration is recorded as an ACTION (a command, not a file), so it needs `--run-actions`:

```
./scripts/rollback-agents-wizard.sh -c claude --run-actions --yes
```

That also restores `~/.claude/settings.json`, since the Glyphdown hooks belong to the same component. To unregister the MCP server *only*, run the recorded undo command yourself:

```
claude mcp remove lumen -s user
```

MiMo Code has no component of its own because there is no MiMo-specific file: it **inherits** MCP servers from `~/.claude.json`. Undoing it therefore means removing the mirror, which lives in the `claude` component:

```
jq 'del(.mcpServers.lumen)' ~/.claude.json > /tmp/cj &&
mv /tmp/cj ~/.claude.json
mimo mcp list | grep lumen      # should now find nothing
```

Note that a full `-c claude` rollback restores `~/.claude.json` byte-exactly, which removes the mirror as a side effect — along with the Glyphdown hooks and the Claude Code registration.

Restart the agent afterwards — MCP servers are launched at session start.

What is stdio vs serve?

`stdio` is Lumen’s MCP subcommand. `serve` does not exist in Lumen.

```
$ lumen serve
Error: unknown command "serve" for "lumen"
Run 'lumen --help' for usage.
```

`lumen stdio` starts the MCP server on stdin/stdout: the agent launches the process and speaks MCP over the pipe. There is no long-running daemon and no port to configure — one server process per agent session, started and stopped by the agent.

That is why the MCP config the wizard writes for Kimi and Qwen Code looks like this:

```
{
  "mcpServers": {
    "lumen": { "command": "/home/<you>/.local/bin/lumen",
              "args": ["stdio"] },
    "codegraph": { "command": "codegraph",
                  "args": ["serve"] }
  }
}
```

Opencode's schema differs — a `.mcp` object with the command as an array — but the subcommand is the same:

```
{
  "mcp": {
    "lumen": { "type": "local", "command": ["/home/<you>/.local/bin/lumen", "stdio"], "enabled": true }
  }
}
```

and Claude Code is registered through its own CLI:

```
claude mcp add-json lumen '{"command":"/home/<you>/.local/bin/lumen","args":["stdio"]}' -s user
```

Note the asymmetry: `codegraph serve` is correct for CodeGraph; `lumen serve` is not. Tests **A3**, **A4** and **D1** pin this down.

If an agent is still configured with `"args": ["serve"]` for Lumen, its MCP server fails to start on every session — see [TROUBLESHOOTING #2](#).

Do I need sudo?

Only for system-level packages and services. Everything the wizard installs for *you* lands in your own home directory: `~/.local/bin`, `~/.local/share`, `~/.bashrc`, `~/.bash_profile` and the per-agent config directories.

sudo appears in exactly three code paths:

Where	Command	Skipped when
ensure_jq	<code>sudo apt-get install -y jq / sudo yum install -y jq</code>	jq is already installed
ensure_ollama	<code>curl -fsSL https://ollama.com/install.sh \& sh (the installer elevates itself on Linux)</code>	ollama is already installed
ensure_ollama	<code>sudo systemctl enable --now ollama</code>	the ollama service is already active

On macOS both use Homebrew, so no sudo at all.

If you cannot use sudo: install jq however your environment allows, and run the embedding backend yourself instead of as a system service —

```
ollama serve &                                # foreground/user-owned backend
export OLLAMA_HOST=http://<host>:11434          # or point Lumen at an existing one
```

The wizard tolerates all three failing: it prints warnings and continues, and `verify_lumen` tells you what is still broken.

There is a fourth sudo path, and it is deliberately *not* in the wizard. `scripts/ollama-vulkan-remediation.sh --apply` writes `/etc/sysconfig/ollama` and runs `sudo systemctl restart ollama`. It lives in a separate script behind an explicit subcommand because restarting a system service and changing host-wide GPU behaviour is a decision an installer has no business making quietly. Test **A41** asserts the wizard’s own source carries no `systemctl restart`, no `/etc` write and no sudo in

the embedding path; the wizard only names the commands, under ACTION REQUIRED. --check, --verify and scripts/lumen-index-doctor.sh need no privileges at all (test I14).

Why does command -v ashlr never find anything?

Because there is no ashlr binary, by design. ashlr is a Claude Code *plugin*, not a CLI. Its installer clones into ~/.claude/plugins/cache/ashlr-marketplace/ashlr/ and deliberately creates no executable and does not touch settings.json.

An earlier revision of the wizard ran check_command ashlr and consequently printed a permanent, misleading for software that was installed and fine. It now verifies the plugin **directory** instead, and reports it in its own summary section:

```
Claude Code plugins / optional tools:
    ashlr plugin installed (activate with /plugin install inside
Claude Code)
```

Tests A42 (no PATH probe) and A43 (directory check present) stop that regressing.

Downloading it is not the same as activating it. Three slash commands inside Claude Code finish the job, plus one to confirm:

```
/plugin marketplace add ashldrai/ashlr-plugin
/plugin install ashlr@ashlr-marketplace
/reload-plugins
/ashlr:ashlr-status
```

The wizard can tell whether you have done them: plugins/cache/<mkt>/ means the installer cloned the bits, while plugins/marketplaces/<mkt>/ is written by Claude Code itself when a human runs /plugin marketplace add. Only the second means the plugin is wired in — so the ACTION REQUIRED entry appears until you have actually done it, then disappears on the next run. See [ACTION-REQUIRED.md](#).

Optionally, /ashlr:ashlr-genome-init then builds a project genome at <project-root>/.ashlrcode/genome/ — 15 sections across vision/, milestones/, strategies/ and knowledge/, indexed by manifest.json. It is optional, it lives inside your project, and no script creates it.

The index was audited and declared trustworthy. Why is there a corruption report?

Because the audit's conclusion was wrong, and understanding why is the most useful thing in this documentation set.

LUMEN-INDEX-INTEGRITY.md decoded every vector in the index and checked for NaN, Inf, all-zero values and L2 norms outside 0.99–1.01. It found none — 0, 0, 0, 0, with every norm inside 0.999999–1.000001 — and concluded TRUSTWORTHY.

Every one of those measurements is correct and reproduces exactly. **All four are per-vector tests, and the failure mode that actually occurred is invisible to per-vector tests.** On a GPU/Vulkan embedding path, ollama can return the *previous* vector: well-formed, 768 dimensions, unit norm — simply not the vector for the text you sent. It passes all four checks by construction.

The one check that would have caught it, aggregate distinctness, was run on blocks 1–4 only (4,096 of 35,717 vectors). The corruption lived in blocks 28–29: **758 identical vectors covering 695 distinct texts across 55 files.**

A vector can be perfectly well-formed and still be the wrong vector. Health is not a property you can establish one vector at a time.

The settled verdict is INDEX-CORRUPTION-RECONCILIATION.md. Use that one to decide whether an index can be trusted. LUMEN-INDEX-INTEGRITY.md is kept **unedited** as a historical record of how a careful audit reached a wrong conclusion; the same report also falsified the “large chunks are the trigger” hypothesis (the largest chunk in the corrupted range was 2,832 characters — the operative quantity is the **batch total**, and Lumen sends 32 chunks per request).

What follows from it in the code: `scripts/lumen-index-doctor.sh` tests aggregate distinctness alongside the conventional checks (I8, I9), and both backend probes send a real 32-text batch and require 32 distinct vectors back (I7). Check your own index with:

```
./scripts/lumen-index-doctor.sh "$PWD"      # exit 1 = corruption  
found
```

What is the ACTION REQUIRED section at the end?

The steps a shell script genuinely cannot perform: Claude Code slash commands (they run inside a Claude Code session) and privileged host changes (they need your `sudo`).

The alternative — skipping them quietly — is how a wizard ends up printing a green summary for work nobody did. So the wizard detects each pending step from your machine's actual state, prints them, writes `<project-root>/MANUAL-STEPS.md`, and on a terminal pauses for Enter so the list cannot scroll past unread.

- **It is not a checklist.** Nothing is ticked off. Do a step, re-run the wizard, and the entry is gone because the condition that raised it no longer holds.
- **MANUAL-STEPS.md is overwritten every run** and is not in the rollback manifest. `rm` it when you are done, or `.gitignore` it.
- **WIZARD_NONINTERACTIVE=1 skips the pause** (the section still prints and the file is still written). The pause is also skipped automatically when stdin is not a terminal.
- **This is the wizard's only blocking prompt.** A `sudo` password prompt comes from `sudo`.

Tests **A48**, **A49**, **A50**. Full detail: [ACTION-REQUIRED.md](#).

Does the wizard index my project?

Not by default. It sets up the launcher, `PATH`, completions, the embedding backend and the MCP configs. If a bounded probe (`timeout 20 lumen search "x" -p "$PROJECT_ROOT" --summary -n 1`) cannot confirm a usable index, it raises an ACTION REQUIRED step naming the commands:

```
./scripts/lumen-reindex.sh "/path/to/project"  
./scripts/lumen-index-doctor.sh "/path/to/project" # verify  
afterwards
```

A bare `lumen index "/path/to/project"` still works and is what those wrap. The wrapper is better for a long unattended run — it refuses to start on an embedding backend known to write corrupt vectors, and retries around transient faults instead of dying halfway and printing

Lumen's usage text, which reads like *you* mistyped the command. The doctor is the part people skip: **a successful index is not necessarily a correct one** (see [the index-audit question](#) above).

Indexing is incremental, so re-running after large external changes (a big rebase, a generated-code drop) is cheap. Use `-f / --force` when you want a full rebuild — and note it is **mandatory** after a corruption event, because a corrupted file keeps a valid content hash and an incremental run skips it forever.

If you want the wizard to do it, opt in:

```
WIZARD_INDEX_PROJECT=1 ./scripts/setup-agents-wizard.sh
```

That enables **Step 8: Indexing This Project**, which builds or refreshes *both* indexes for the project root:

- **CodeGraph** — `codegraph sync "$PROJECT_ROOT"` when `.codegraph/codegraph.db` already exists, otherwise `codegraph init "$PROJECT_ROOT"`, which creates `.codegraph/`.
- **Lumen** — `lumen index "$PROJECT_ROOT"`.

Either half is skipped with a warning if its CLI is missing. Without the variable, Step 8 prints one line — `Skipping project indexing (set WIZARD_INDEX_PROJECT=1 to build indexes)`. — and the Step 8 header never appears at all.

It is opt-in for a reason: installing an indexer is fast, running one is not. A first Lumen pass on a large repository is CPU-bound on the embedding backend and can take hours, and you rarely want that happening inside a setup script you are watching.

The wizard never runs `codegraph index`. That command deletes the existing database before rebuilding it, which would throw away a good index on every re-run. `sync` is the incremental path, and test **A27** fails the build if `codegraph index` reappears.

Why did the wizard show me an ollama command instead of just running it?

Because running it restarts the ollama service, and that aborts every embedding request in flight. If an index run is part-way through, it loses that work. The wizard is not entitled to make that trade for you.

Step 7 asks `scripts/ollama-tune.sh` how ollama is managed on **this** host and what concurrency this host's CPU, RAM and loaded model justify, shows you the answer, and puts the delegate's exact commands into ACTION REQUIRED. To let it apply them:

```
WIZARD_TUNE_OLLAMA=1 ./scripts/setup-agents-wizard.sh
```

Even then it asks first, and it refuses while an indexer is running — or while it cannot tell whether one is (no `pgrep`), because “cannot tell” is not “idle”. If it does apply, the change lands in the rollback manifest as `bash <path> --revert`, alongside a NOTE recording that the aborted in-flight requests are not something `--revert` can bring back.

It will never print an example number. If the tuner is missing, cannot reach a verdict, or there is no ollama at all, you get a step that says nothing was measured — not a plausible-looking value that was never computed on your machine.

One subtlety worth knowing, because it bit the first version of this step: `ollama-tune.sh` exits 1 for “*a real problem / action required*”, which is exactly when it **does** have a recommendation. 0 means nothing to apply and 2 means it could not determine. The wizard maps all three explicitly — a non-zero exit is not read as “the tuner broke”.

Step 9 said the provider CI status is “UNVERIFIED”. Is that a failure?

No, and it is deliberately not a pass either. `scripts/verify-provider-ci.sh` answers with three values, and the wizard keeps all three: 0 nothing found, 1 confirmed provider-side triggering, 2 could not determine. Exit 2 — and a timeout, an unexpected status, a missing script, or a missing `gh` — is reported as `COULD NOT BE DETERMINED / Reporting UNVERIFIED`. This is NOT a pass.

The usual cause is that `gh` is not authenticated or the token lacks admin scope on a repository. Check with `gh auth status`, then run the verifier again.

This matters because everything else in this repository checks **files**, and a file-level check structurally cannot see an org-default required workflow, a branch-protection required check, the GitHub Pages

source setting, or a provider-side scheduled export. Those remain operator-only to *change* — but their status is now measured on demand rather than asserted in prose.

Neither index is recorded in the rollback manifest — they are tool-owned data, not wizard edits. Remove them with `rm -rf .codegraph` and `lumen purge "/path/to/project"`.

What does the wizard do about telemetry?

It turns it **off**, for the tools it installs, as the first act of Step 3 (`configure_telemetry_optout`). Opt out of the opt-out with `WIZARD_KEEP_TELEMETRY=1`, which leaves every setting below exactly as it found it and prints

```
WIZARD_KEEP_TELEMETRY set - leaving telemetry settings alone.
```

Tool	What is set	Where
Any tool honouring the convention	<code>export DO_NOT_TRACK=1</code>	A third managed block in <code>~/.bashrc</code> , <code># >>> telemetry opt-out (managed by Claude Code)</code> <code>>>></code>
CodeGraph	<code>export CODEGRAPH_TELEMETRY=0,</code> plus <code>codegraph telemetry off</code> (which also deletes its buffered data)	The same <code>~/.bashrc</code> block; the CLI persists its own state in <code>~/.codegraph/telemetry.json</code>
Qwen Code	<code>.usageStatisticsEnabled = false</code>	<code>~/.qwen/settings.json</code> , via a <code>jq merge</code> that preserves your other keys
Lumen	nothing — it ships no telemetry	—

That last row is the point of stating it explicitly: Lumen’s binary was checked and contains **zero analytics strings**, so there is no switch to flip and the wizard does not pretend otherwise. The summary does not restate that as a past finding either — it re-runs strings against the resolved binary on every run and reports what it just saw: Lumen

ships no telemetry (probed /path/to/lumen-linux-amd64 just now), or
Lumen binary contains analytics-like strings - inspect <path> if
that ever changes, or Lumen telemetry not probed (strings(1)
unavailable) when it cannot check.

None of these values was guessed. DO_NOT_TRACK is the cross-tool
convention (<https://consoledonottrack.com>) and was confirmed by
grepping the @colbymchenry/codegraph bundle; usageStatisticsEnabled
was found in the installed @qwen-code/qwen-code chunks.

Both exports also apply to the wizard's own process, so the rest of the
run is covered without waiting for a new shell.

Two practical notes:

- ~/.qwen/settings.json is only touched when it **already exists** and
jq is available, and only when the flag is not already false —
otherwise you get Qwen usage statistics already disabled.
(Test H7 guards a subtle bug here: jq's // operator returns its right-
hand side for false as well as null, so reading the flag with //
"unset" made a correctly-disabled setting look unset.)
- The ~/.bashrc block and the Qwen edit are both in the rollback
manifest (components shell and qwen). codegraph telemetry off is
an out-of-band CLI call with no ACTION row — re-enable it yourself
if you want it back.